

Final project

Alberto Tarable, Marco Ghirardi

alberto.tarable@polito.it, marco.ghirardi@polito.it

RULES



**POLITECNICO
DI TORINO**

What to deliver?

- The delivery should be an archive containing:
 - the slx file(s) of the Simulink model(s)
 - the Matlab script(s) calling such model(s)
 - the report (as a pdf file)
 - any other file you may deem important
- The report should concisely explain the work done, how the program works, the results, etc.
 - Chosen editor (Word, LaTeX, etc.) does not matter



When to deliver?

- Before the day of the session you subscribe to
 - E.g., if you subscribe for the January 29th session, deliver by January 28th
 - If you then do not show up at (or do not pass) the exam, the delivery will be valid for the whole year
 - Notice: to get the full 5 points of the exam, you need to submit by January 28th , otherwise the maximum grade is 4 points



Grading criteria

- We evaluate:
 - the correspondence to the description
 - the correctness of the implementation
 - the organization of the program
 - the completeness of the implementation
 - the quality of the report

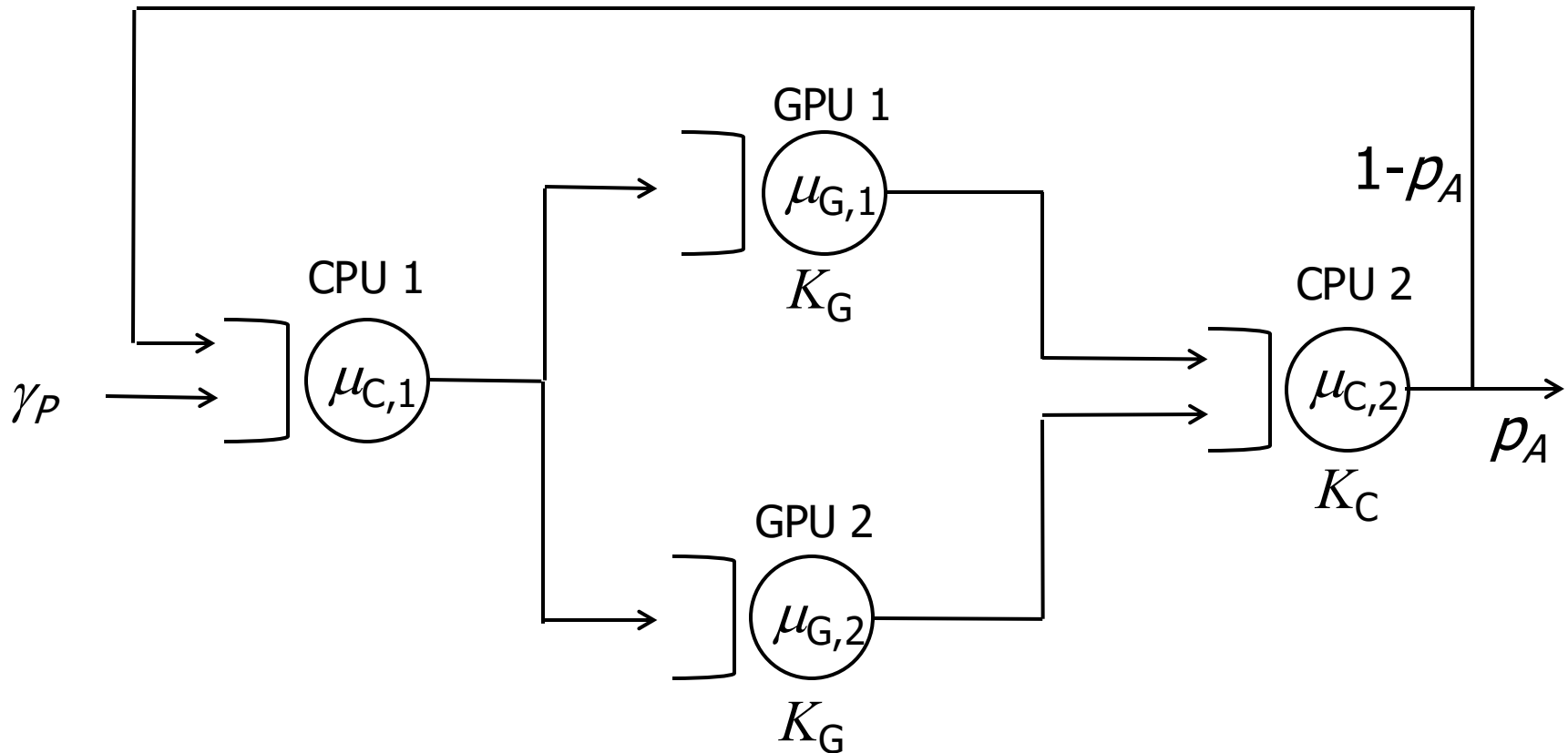


THE PROBLEM



**POLITECNICO
DI TORINO**

Small language model

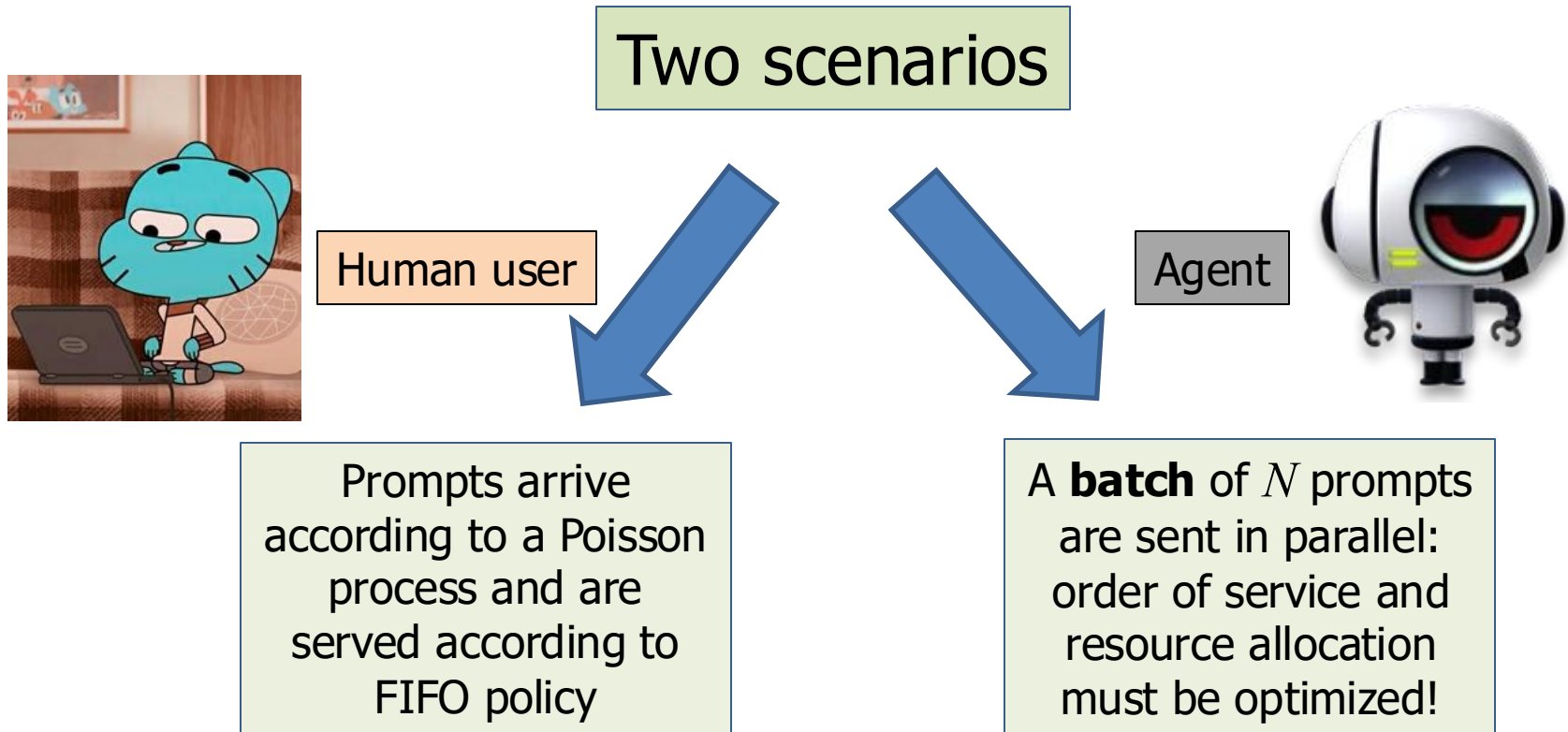


Small language model

- CPU 1: M/M/1 queue
- CPU 2 and GPUs: M/M/1/K
 - K_C capacity of CPU 2
 - K_G capacity of each of the two GPUs
- Exponentially distributed service times, with rates:
 - $\mu_{C1} = \mu_{C2} = 8$ prompts / minute
 - $\mu_{G1} = \mu_{G2} = 5$ prompts / minute



Small language model



Human user scenario

- Arrival rate $\gamma_p = 5$ prompts/ minute, $p_A = 0.7$
- To do:
 - Implement the system in Simulink
 - Find the minimum capacities that make the network ergodic when:
 - Prompts are sent with the same probability to the two GPUs
 - Prompts are sent to the GPU with the lower load (number of prompts)
 - GPUs are unified in a M/M/2/K queue with capacity $2K_G$



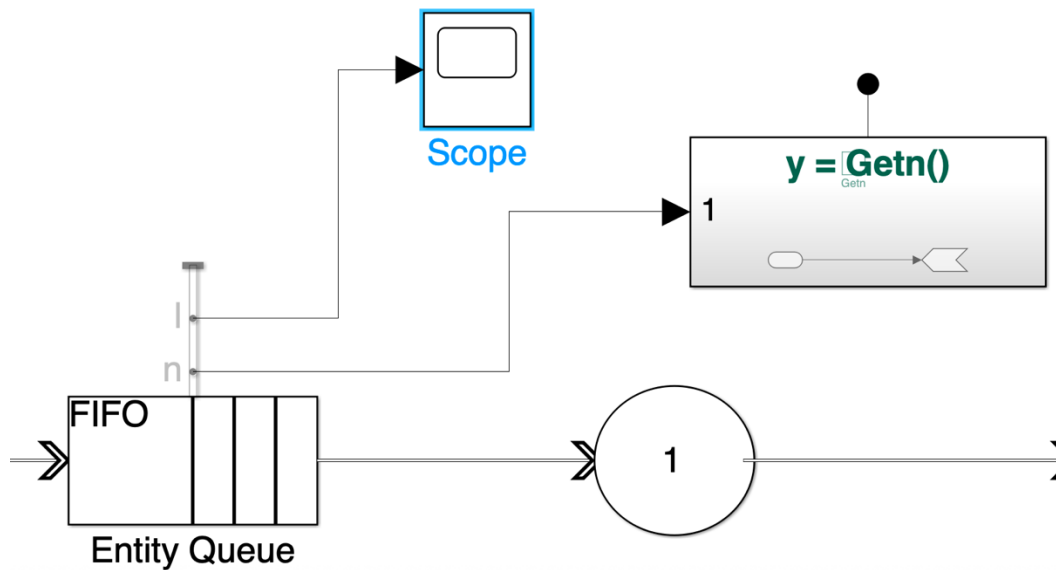
Agent scenario

- $K_C = K_G = \infty, p_A = 1$
- To do:
 - Implement the Simulink system, in the following variants
 - The prompts are served in random order
 - A linear programming model provides the order of service and the assignments of each prompt to GPU 1 or GPU 2, to minimize the makespan (time to answer all prompts)
 - Compare the two variants for $N = 5, 7, 9$



Suggestions

- In the human user scenario, the number of entities (prompts) in a queue can be seen by connecting the corresponding statistics to a Simulink function



Suggestions

- In the agent scenario:
 - you may want to set a limit to the maximum time the solver take to find the optimal solution (especially for N large)
 - set a seed in order to be able to repeat the experiments on the same data with and without optimizer

